

Selenium IDE



Contents

- Introduction to Selenium IDE
- Selenium IDE - Features
- Benefits of Selenium IDE
- Selenese Commands
- Limitations of Selenium IDE
- How to Install Selenium IDE for Firefox and Chrome
- Test Automation using Selenium IDE

Why do we need selenium waits?

Introduction

- Selenium IDE (Integrated Development Environment) is primarily a record/run tool that a test case developer uses to develop Selenium Test cases.
- Selenium IDE is an easy-to-use tool from the Selenium Test Suite and can even be used by someone new to developing automated test cases for their web applications.
- One does not require any special setup to get started with Selenium IDE.
- You just need to add the extension of your specific browser. Selenium IDE provides you with a GUI (Graphical User Interface) for easily recording your interactions with the website.

Why do we need selenium waits?

Introduction

- After entering the base URL for the project, the recording begins, and all the interactions with the website are recorded and categorized into three main categories as shown in the above Screenshot:

- * **Command**

- * **Target**

- * **Value**

Why do we need selenium waits?

Selenium IDE Features

- There are several features provided in the IDE under the toolbar, using which one can control the execution of test cases:
 1. Speed Control – Helps control the speed of test cases
 2. Run All – Allows execution of the entire Test Suite
 3. Run – Runs the currently selected test
 4. Pause/Resume – Allows a user to pause and resume a particular test case
 5. Step – Helps step into each specific command in the test script
 6. Rollup – Helps group all the Selenese Commands together and make them execute as a single operation

Why do we need selenium waits?

Benefits

- Provides you the capability of automatically recording your test cases based upon the interactions with the browser
- Gives developers greater flexibility in executing the test cases. Either the test developer can run the entire test suite consisting of multiple test cases or execute a single test case
- Operates on the basis of the rich set of Selenese commands, which helps the IDE understand what needs to be done
- Allows the test developers to set breakpoints for the purpose of debugging particular test cases
- Test cases can be re-used using the run command. (e.g. allowing you to re-use the logic of login or reload on multiple places in the entire suite)

Why do we need selenium waits?

What is Selenese?

- Selenese is the language used to write Selenium Commands. These Selenese commands are then used to test web-applications. Based on the HTML tags of the UI elements, one can check their existence. Commands help Selenium understand what actions or operations to perform.

Classification of Selenium Commands

- Selenium commands are mainly categorized into three types:
- Actions – Help manipulate or change the state of applications (e.g. click on some link or select an option from a page). While the rest of the page continues parsing, which enhances responsiveness and application performance.

Why do we need selenium waits?

What is Selenese?

- Actions – Help manipulate or change the state of applications (e.g. click on some link or select an option from a page).
- Accessors – Enable verification and storage of the application state (e.g. consider command “storeTextPresent” – if the text is found on the page, then it stores True else stores false).
- Assertions – Help compare expected and actual results. They act like checkpoints and if both the values are equal, only then the test case passes or else it fails. Thus, Assertions help verify whether the state of the application after executing the test case conforms to the desired state (e.g. VerifyText, waitForPageToLoad). Assertions have three modes:
 - Assert
 - Verify
 - WaitFor

Why do we need selenium waits?

Limitations of Selenium IDE

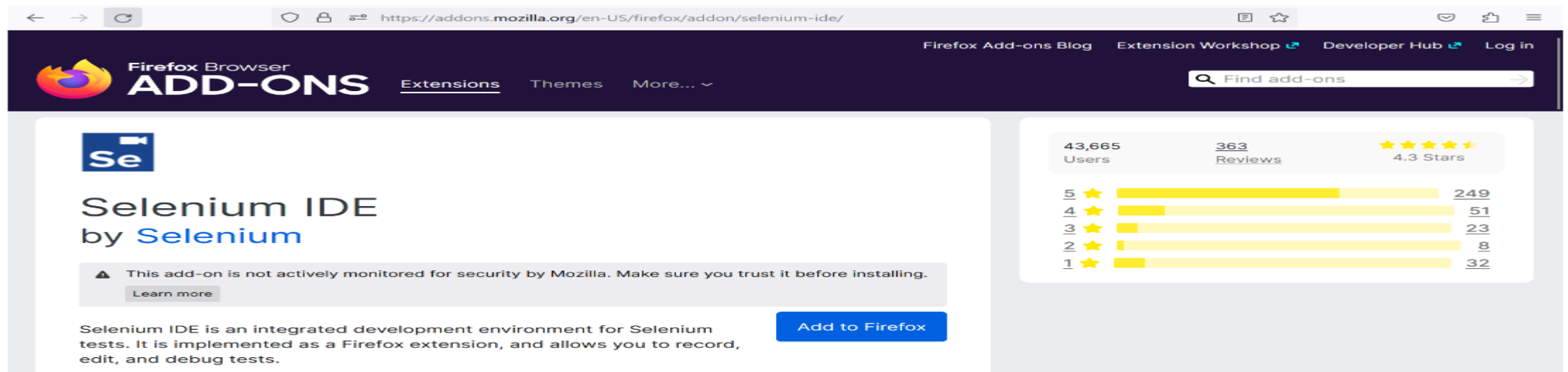
- Not suitable for testing extensive data
- Connections with the database can not be tested
- Cannot handle the dynamic part of web-based applications
- Does not support capturing of screenshots on test failures
- No feature available for generating result reports

Why do we need selenium waits?

How to install Selenium IDE - Firefox

1. Open Firefox browser and navigate to <https://addons.mozilla.org/en-US/firefox/addon/selenium-ide/>.

2. Click on the “Add to Firefox” button which will open a pop up. Then click on the “Add” button.



Why do we need selenium waits?

How to install Selenium IDE

The screenshot shows the Selenium IDE add-on page on the Firefox Add-ons website. The page header includes the Firefox logo and 'ADD-ONS' text. The main content area displays the Selenium IDE logo and title, followed by a warning message: 'This add-on is not actively monitored for security by Mozilla. Make sure you trust it before installing.' Below this is a description: 'Selenium IDE is an integrated development environment for Selenium tests. It is implemented as a Firefox extension, and allows you to record, edit, and debug tests.' On the right side, there is a 'Reviews' section showing a 4.3 star rating and a list of star ratings with their respective counts.

Firefox Add-ons

Selenium IDE
by Selenium

⚠ This add-on is not actively monitored for security by Mozilla. Make sure you trust it before installing.
[Learn more](#)

Selenium IDE is an integrated development environment for Selenium tests. It is implemented as a Firefox extension, and allows you to record, edit, and debug tests.

Add Selenium IDE? This extension will have permission to:

- Access your data for all websites
- Download files and read and modify the browser's download history
- Access browser tabs
- Access browser activity during navigation

[Learn more](#)

[Add](#) [Cancel](#)

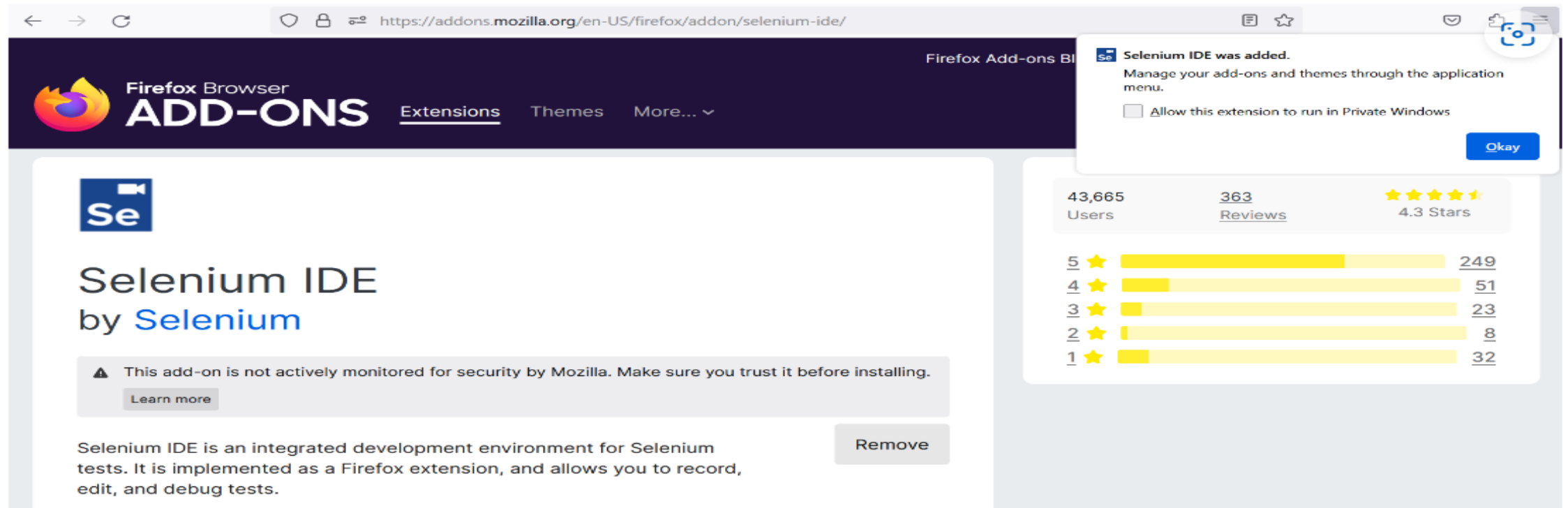
Users Reviews 4.3 Stars

5	★	249
4	★	51
3	★	23
2	★	8
1	★	32

Why do we need selenium waits?

How to install Selenium IDE

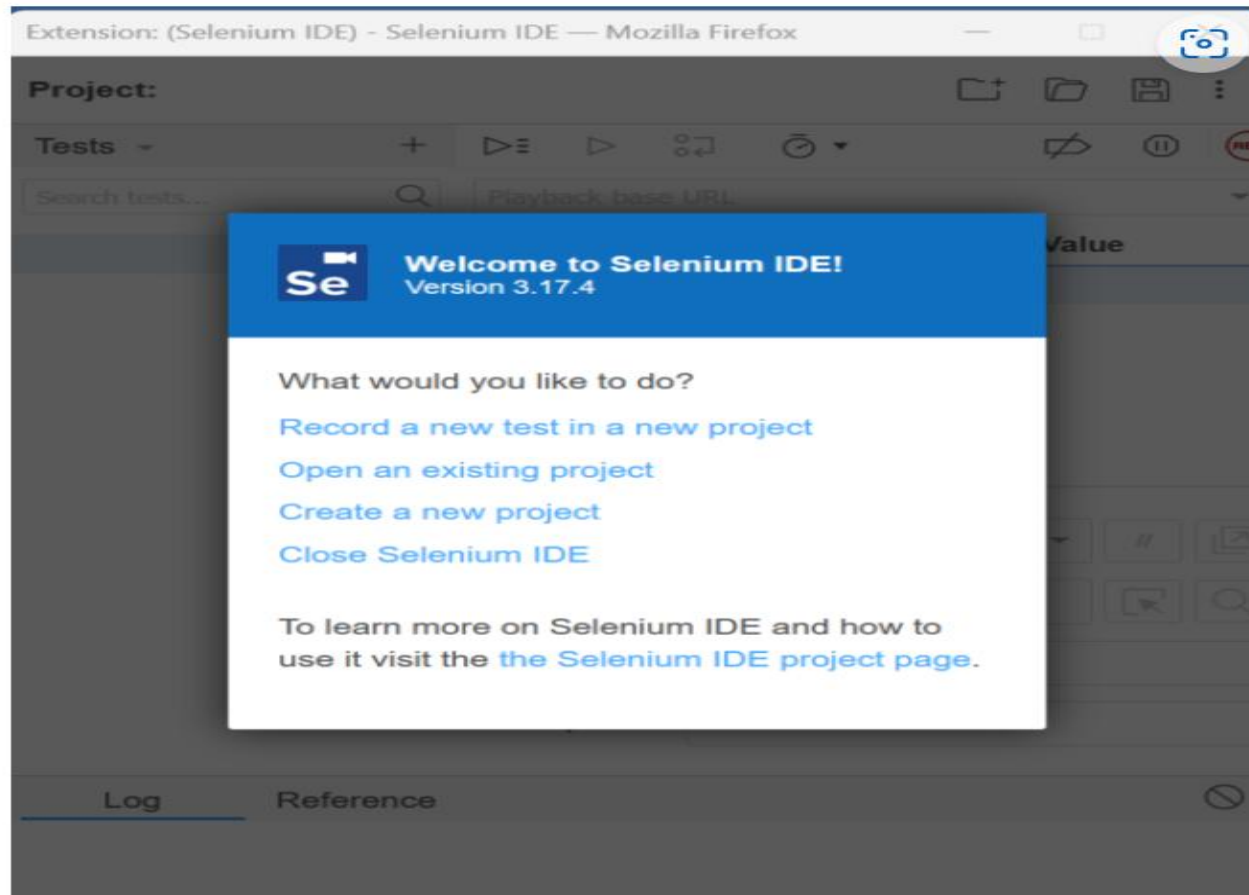
3. Once the installation is complete, a pop up would be displayed saying Selenium IDE was added. Click on the “Okay” button.



Why do we need selenium waits?

How to install Selenium IDE

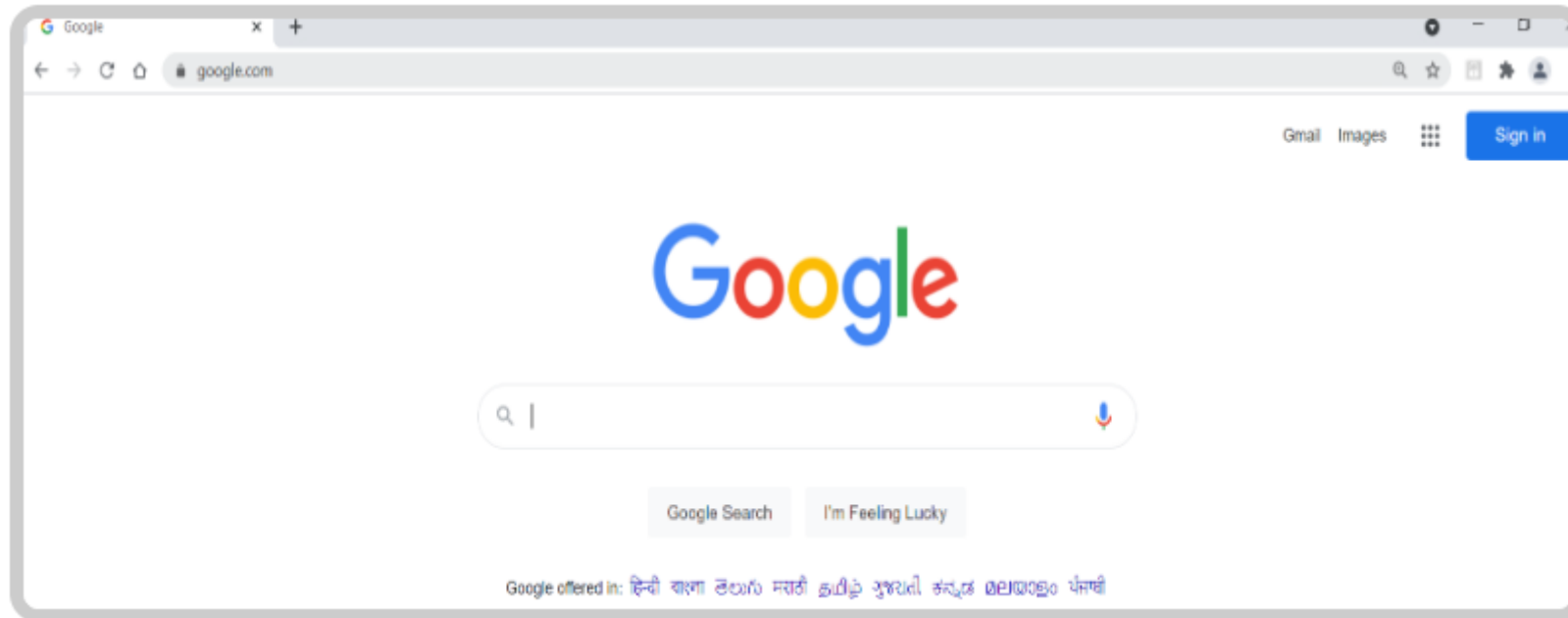
4. Selenium IDE plugin should be visible now on top of the browser. Click on it to open



Why do we need selenium waits?

How to install Selenium IDE - Chrome

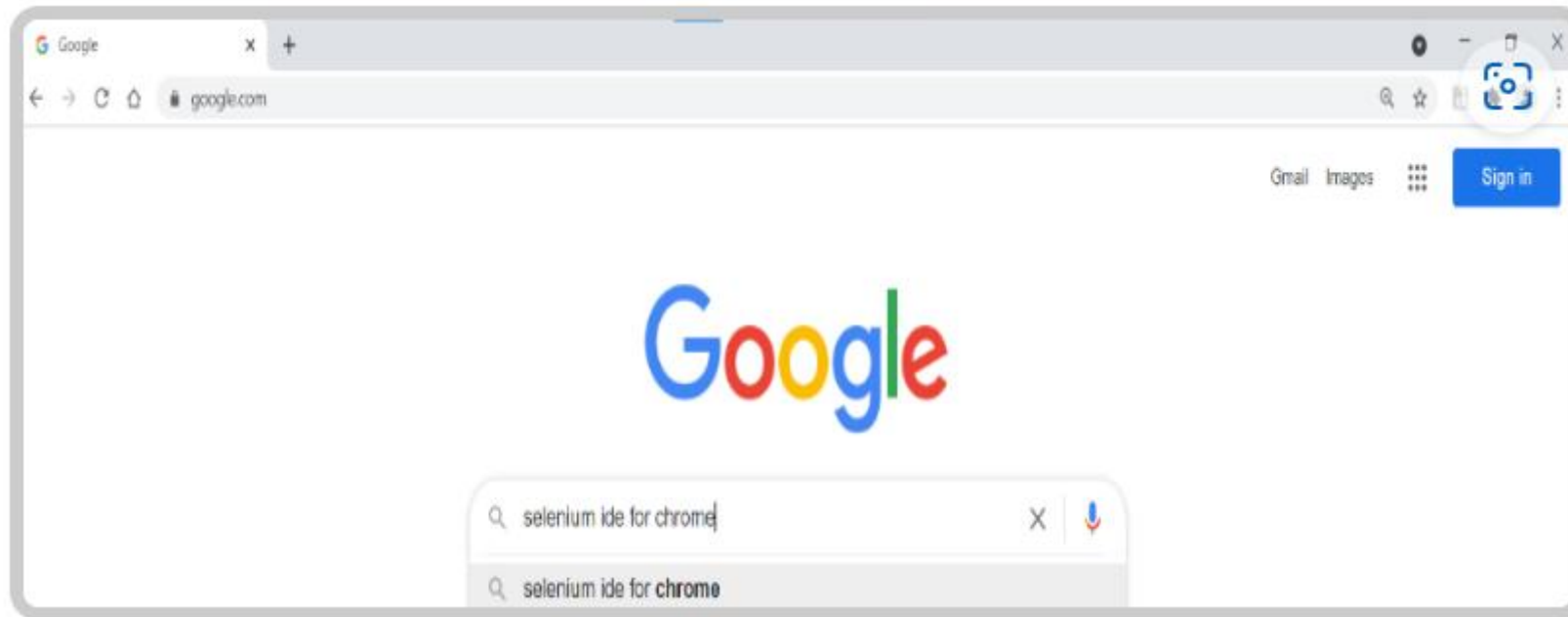
Step 1: Launch the Chrome browser.



Why do we need selenium waits?

How to install Selenium IDE - Chrome

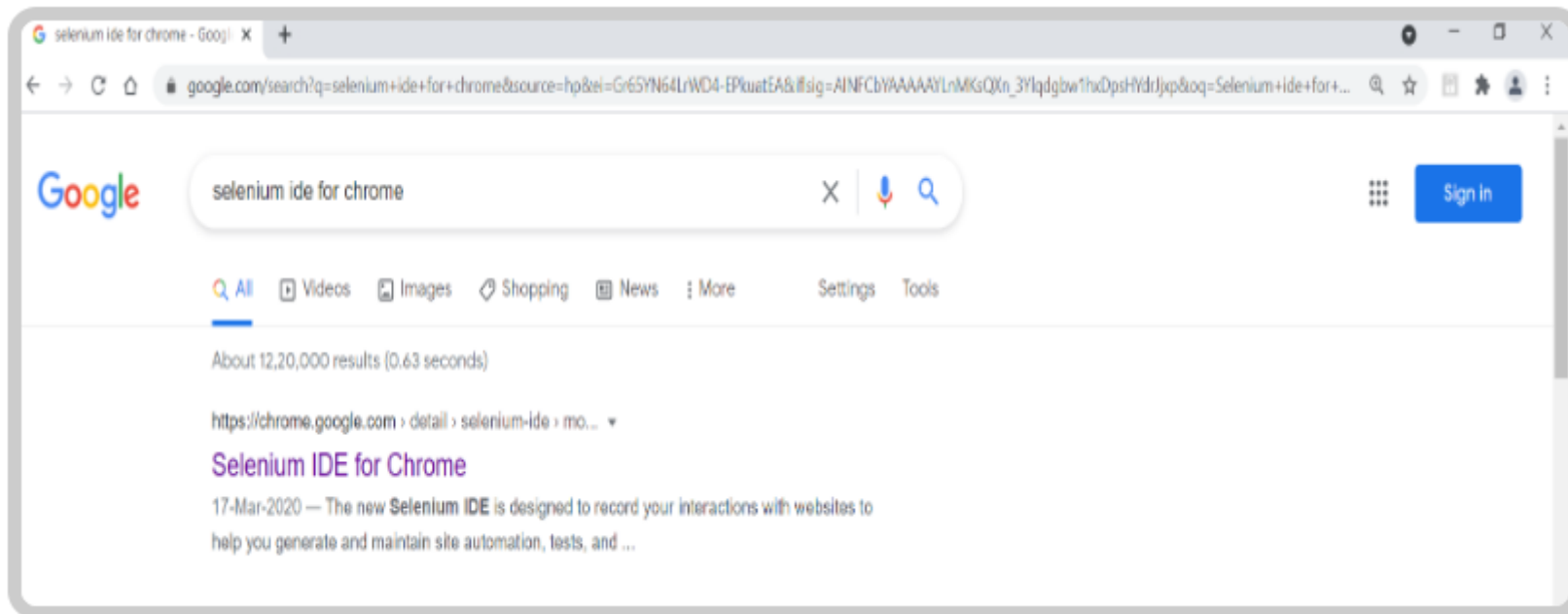
Step 2: Search with the keyword 'Selenium IDE for Chrome' in the browser.



Why do we need selenium waits?

How to install Selenium IDE - Chrome

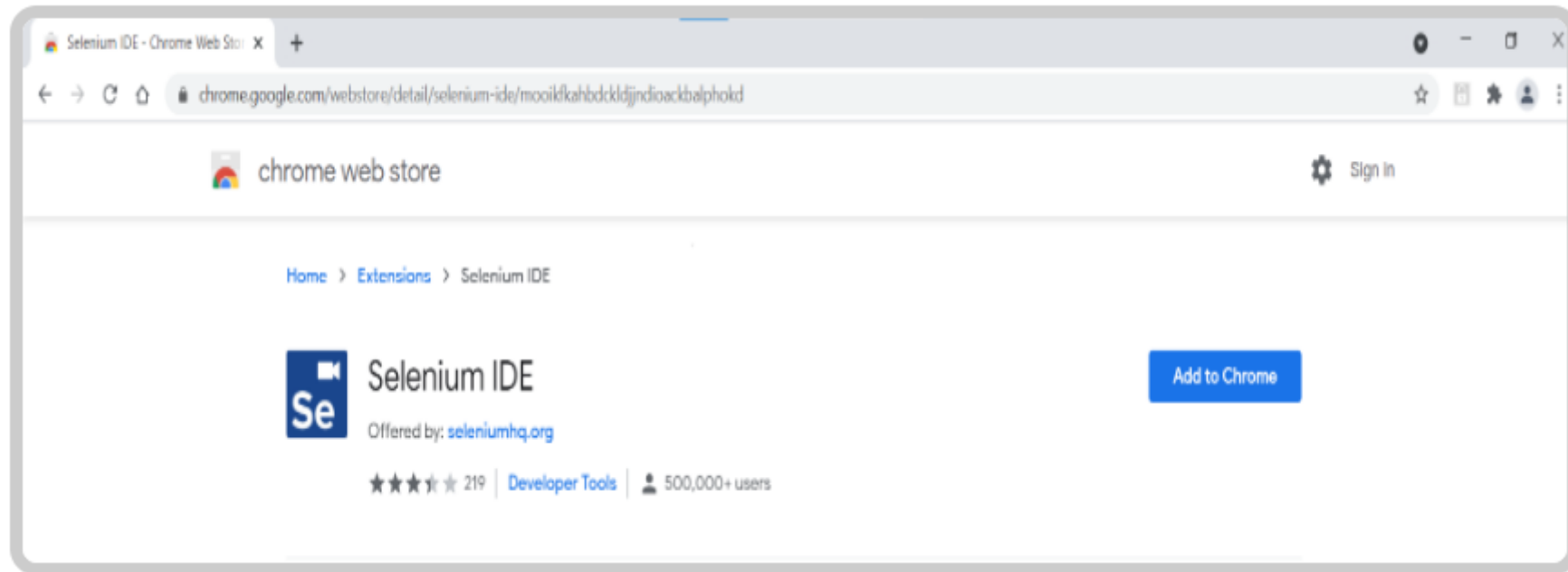
Step 3: Click on the option shown in the below screen.



Why do we need selenium waits?

How to install Selenium IDE - Chrome

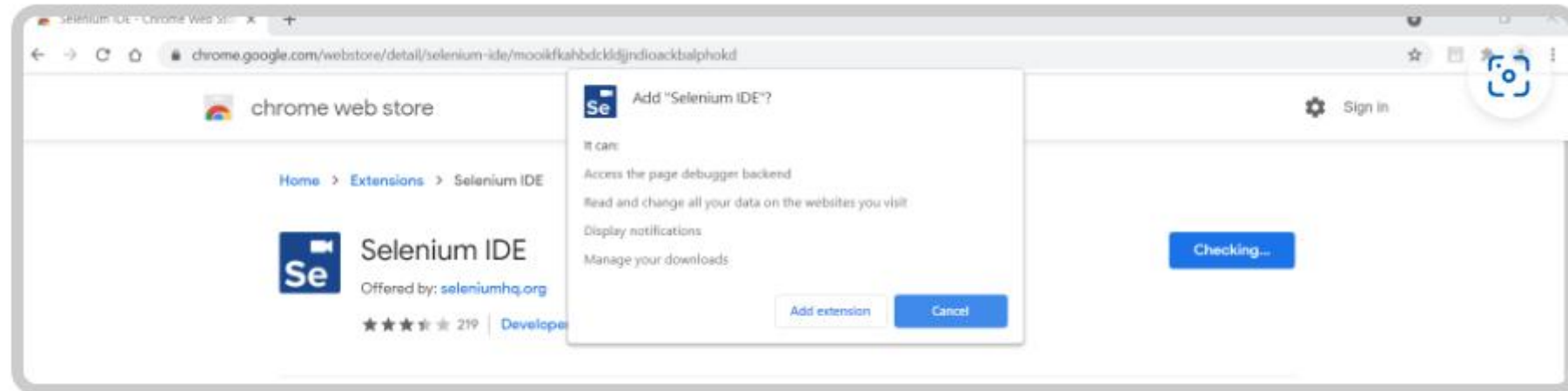
Step 4: Click on the "Add to Chrome" option.



Why do we need selenium waits?

How to install Selenium IDE - Chrome

Step 5: Click on "Add extension."



Why do we need selenium waits?

How to install Selenium IDE - Chrome

Step 6: Restart the Chrome browser once the above is complete.

Step 7: Open Selenium IDE from Chrome and start the testing on any web application, as per the business requirements.

Selenium IDE - Introduction

➤ **Selenium IDE - Introduction :**

- ✓ Selenium IDE is a browser based extension to record and play back.
- ✓ No programming experience is required, though knowledge of HTML and DOM are required.
- ✓ Designed for Firefox, Chrome, IE
- ✓ Recorded generate changed to Java, C#, PHP, Python, Ruby language

Selenium IDE – Features

➤ Menu Bar

- Create Test case and Test Suite
- Save/Open the Test case
- Copy, Paste and Delete the command
- Record and Play, Clipboard Format

➤ Action Bar :



Speed Control: controls how fast your test case runs.

Run All: Runs the entire test suite

Selenium IDE – Features

Action Bar



Runs the Currently Selected test.



Allows stopping and re-starting of a running test case

Allows you to “step” through a test case

Records the user’s browser actions

Selenium IDE – Features - CONTD

➤ Test case pane

- The Command, Target, and Value entry fields display the currently selected command along with its parameters

➤ Log/Reference

- Log -Test case, error messages and information messages are displayed, useful for debugging
- Reference - Reference pane will display documentation on the selenese command

Selenium IDE – Features - CONTD

Features :

The screenshot displays the Selenium IDE interface. At the top, a browser window shows the URL `https://www.google.co.in/`. Below it is a table of commands:

	Command	Target	Value
5	run script	window.scripts(0,372)	
6	run script	window.scripts(0,512,666870117,198)	
7	run script	window.scripts(0,1393)	

Below the table is the command editor with the following fields:

- Command: `type`
- Target: `name=q`
- Value: `hello`
- Description: (empty)

At the bottom is the Log tab, showing a list of actions:

	Log	Reference
	Running Google_Search	12:00:40
1.	open on https://www.google.co.in/ OK	12:00:46
2.	setWindowSize on 1054x660 OK	12:00:46
3.	type on name=q with value hello OK	12:00:47
4.	sendKeys on name=q with value \${KEY_ENTER} OK	12:00:48

Selenium IDE – Features - Exporting

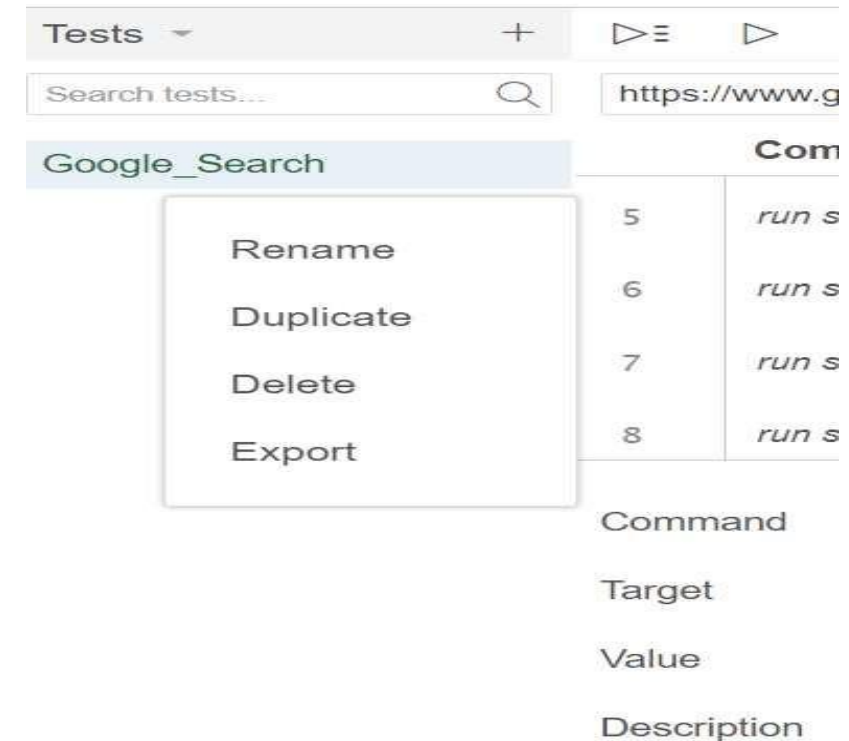
Exporting :

A most interesting feature of Selenium IDE: export to Selenium RC.

Selenium IDE is able to export test cases in the following formats:

- **C# NUnit**
- **C# xUnit**
- **Java JUnit**
- **JavaScript Mocha**
- **Python pytest**
- **Ruby Rspec**

You can export test-cases or the entire test-suite.



Why do we need selenium waits?

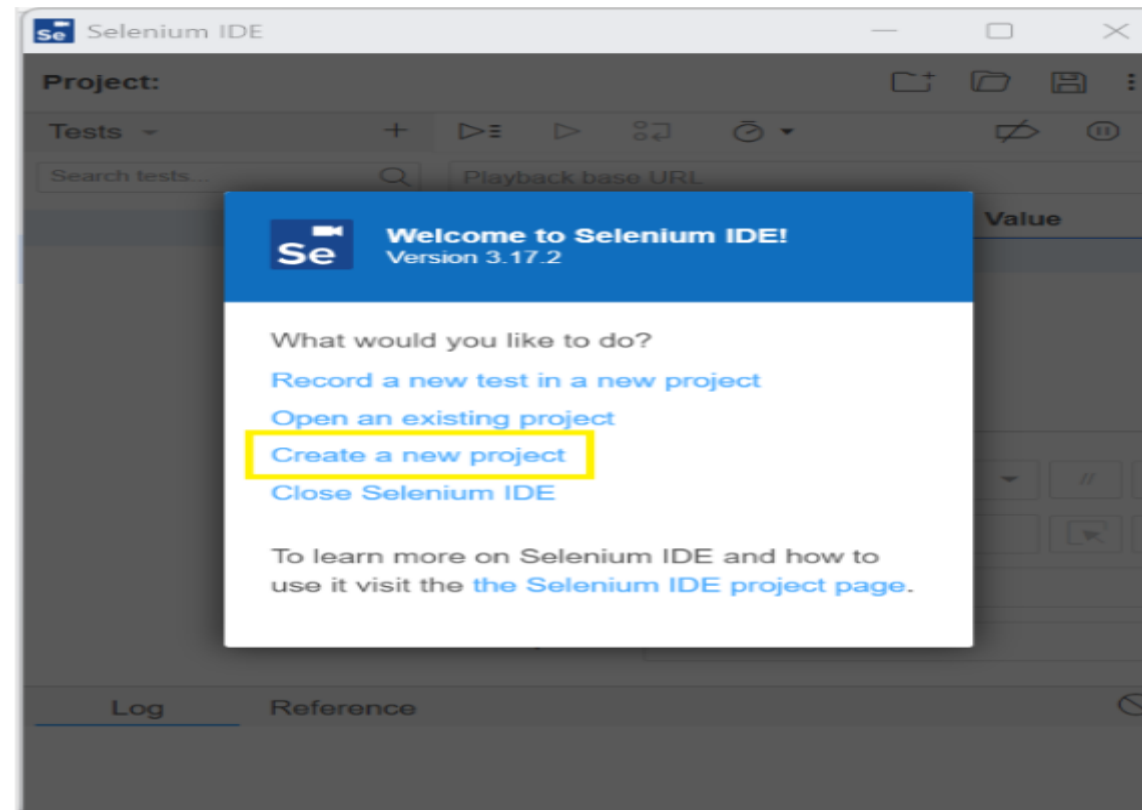
Test Automation using Selenium IDE

- Once we have the Selenium IDE setup on our respective browser, i.e Chrome or Firefox, it is time to see how to record and playback scripts in the IDE.
- We would be using the BStackDemo application to test the checkout process.
 - 1.We would Record the checkout process on the BStack Demo application.
 - 2.Save the test suite.
 - 3.Play back the script
 - 4.Export test script.

Why do we need selenium waits?

Record process in Selenium IDE

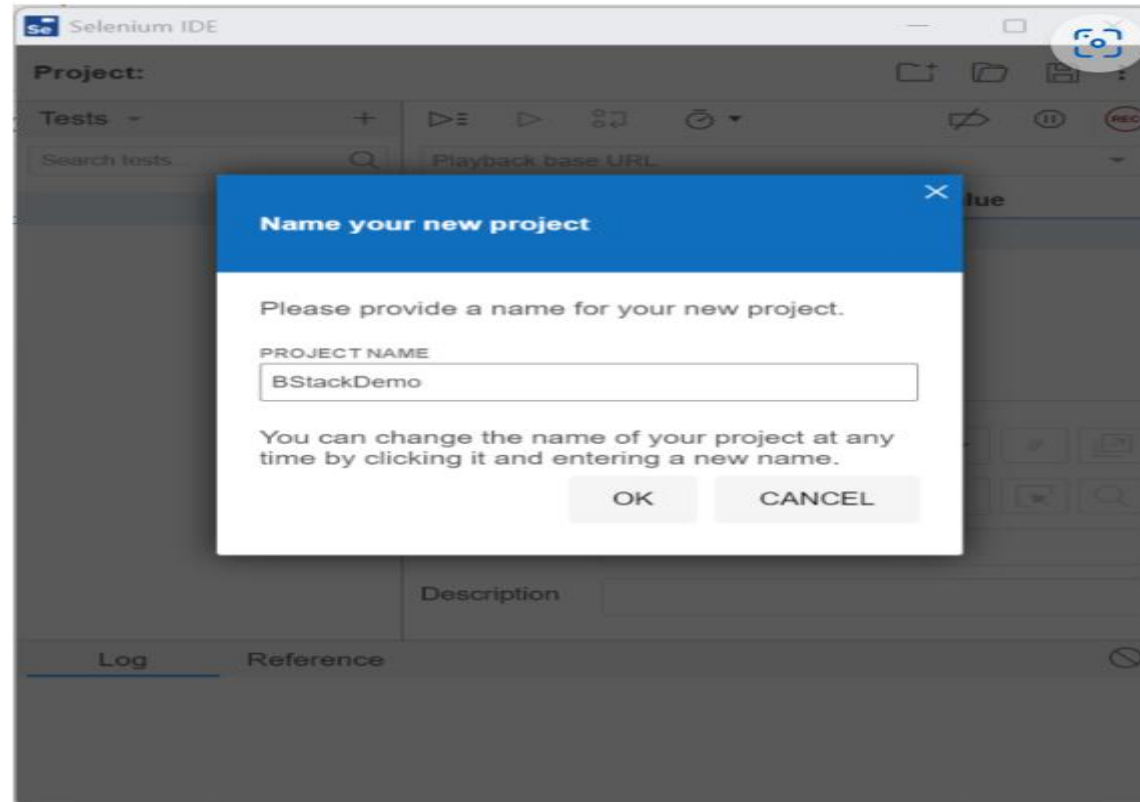
1. Open Chrome/ Firefox browser and click on the Selenium plugin present at the top right corner of the browser to open Selenium IDE.



Why do we need selenium waits?

Record process in Selenium IDE

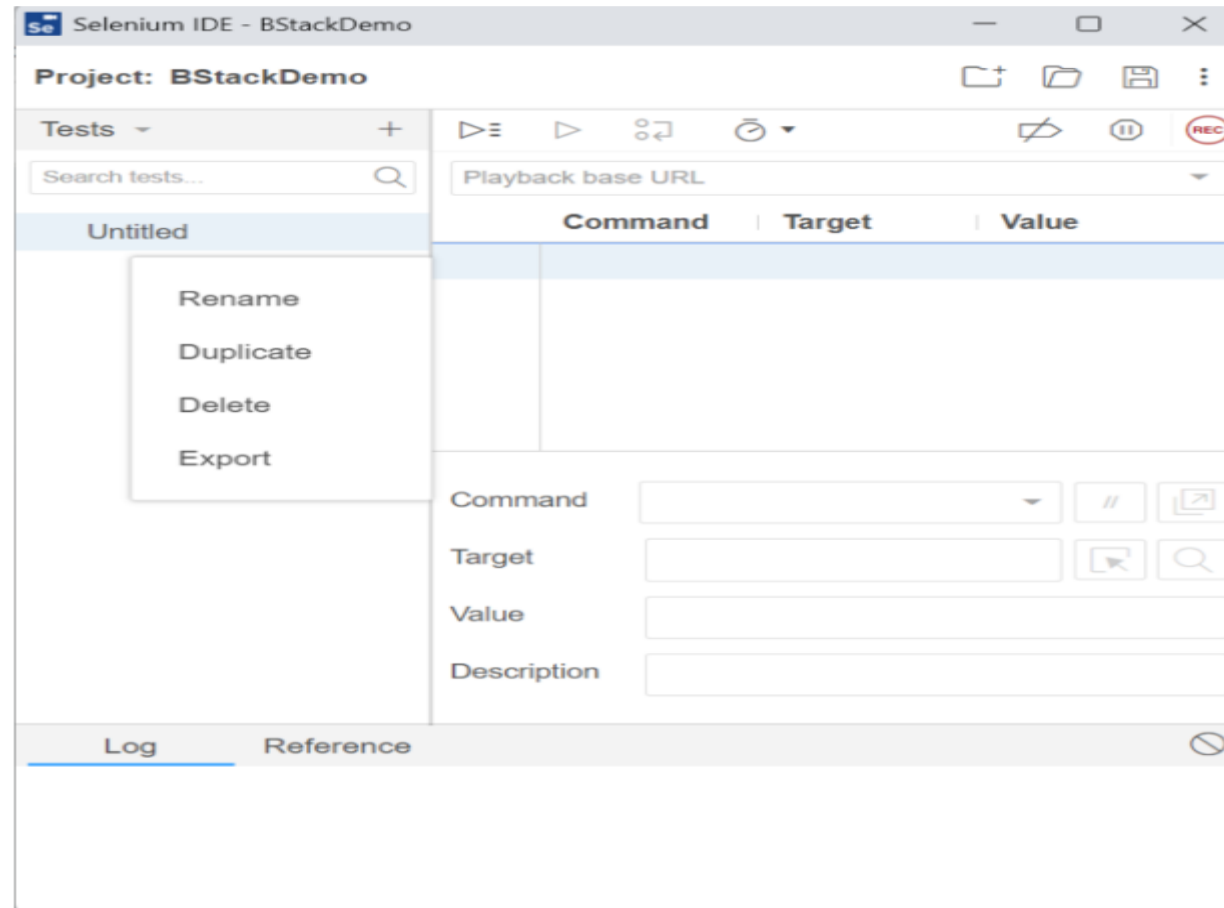
2. Click on Create a new Project.
3. Add the project name and click on the "OK" button.



Why do we need selenium waits?

Record process in Selenium IDE

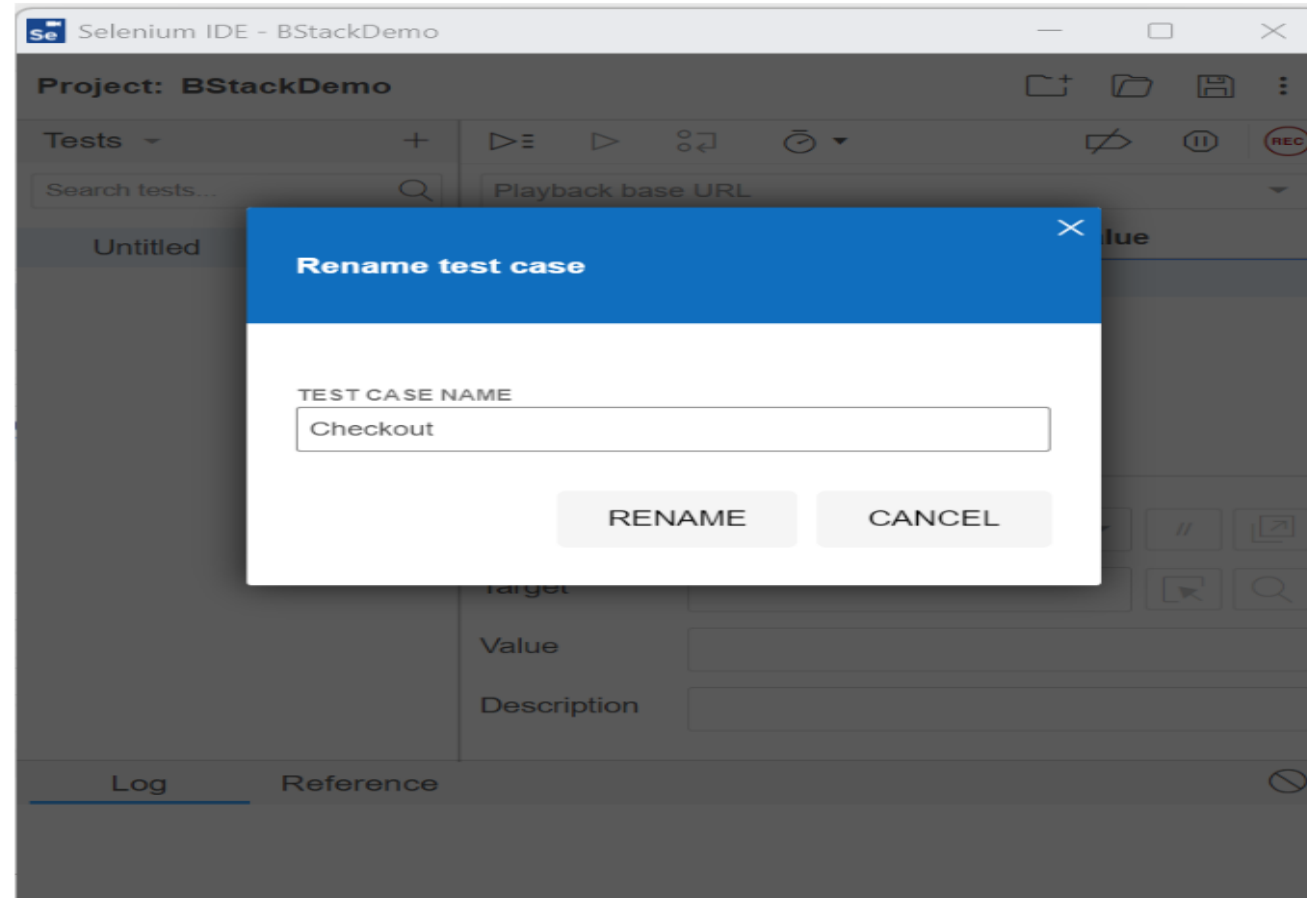
4. Rename Untitled test case to any appropriate test case name.



Why do we need selenium waits?

Record process in Selenium IDE

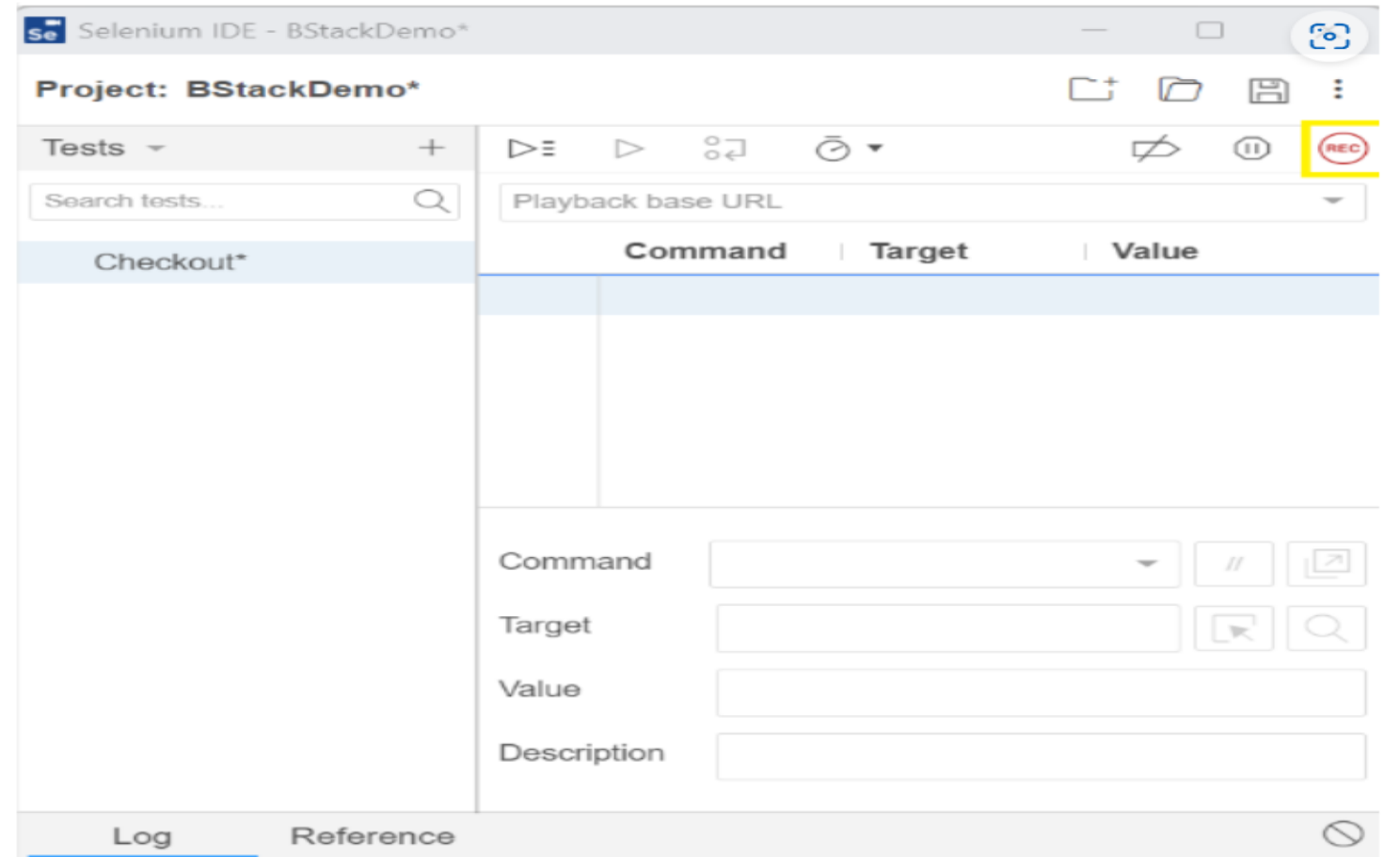
4. Rename Untitled test case to any appropriate test case name.



Why do we need selenium waits?

Record process in Selenium IDE

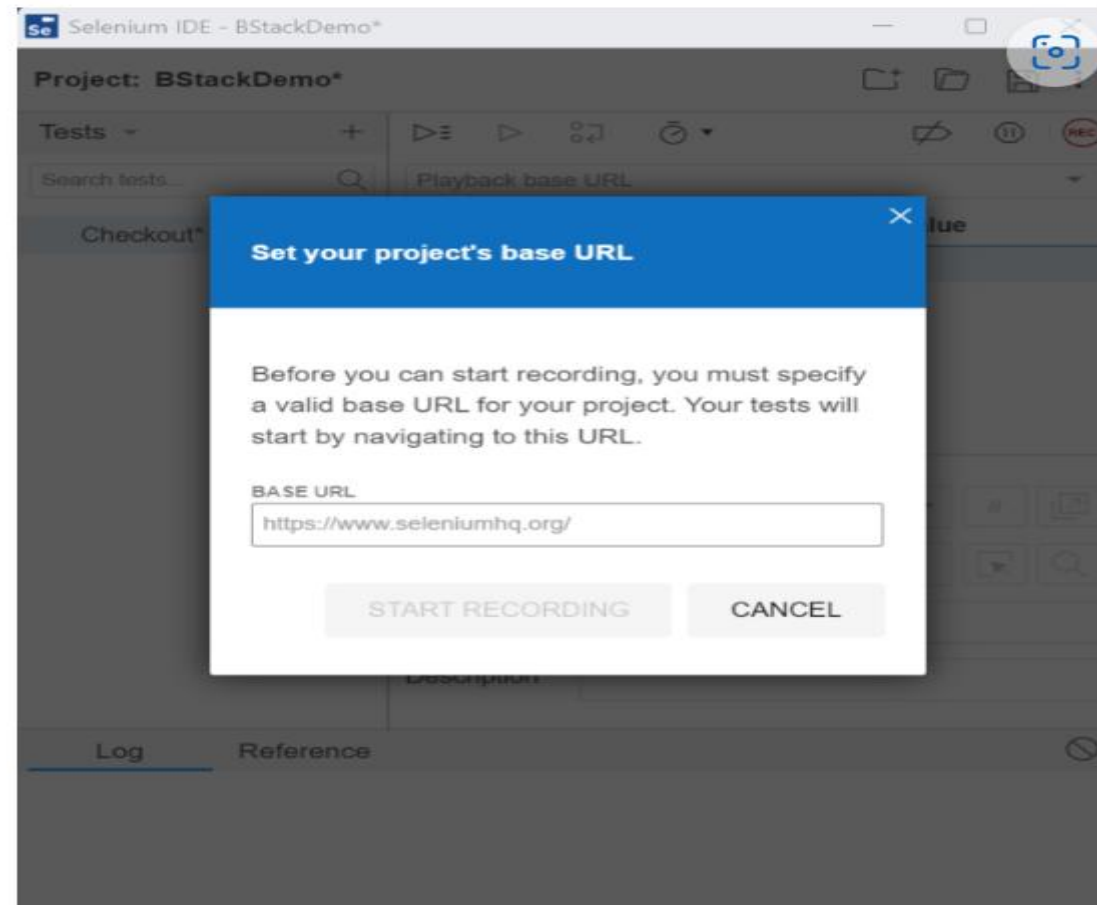
5. Click on the Record button at the top right corner of IDE to start recording the steps.



Why do we need selenium waits?

Record process in Selenium IDE

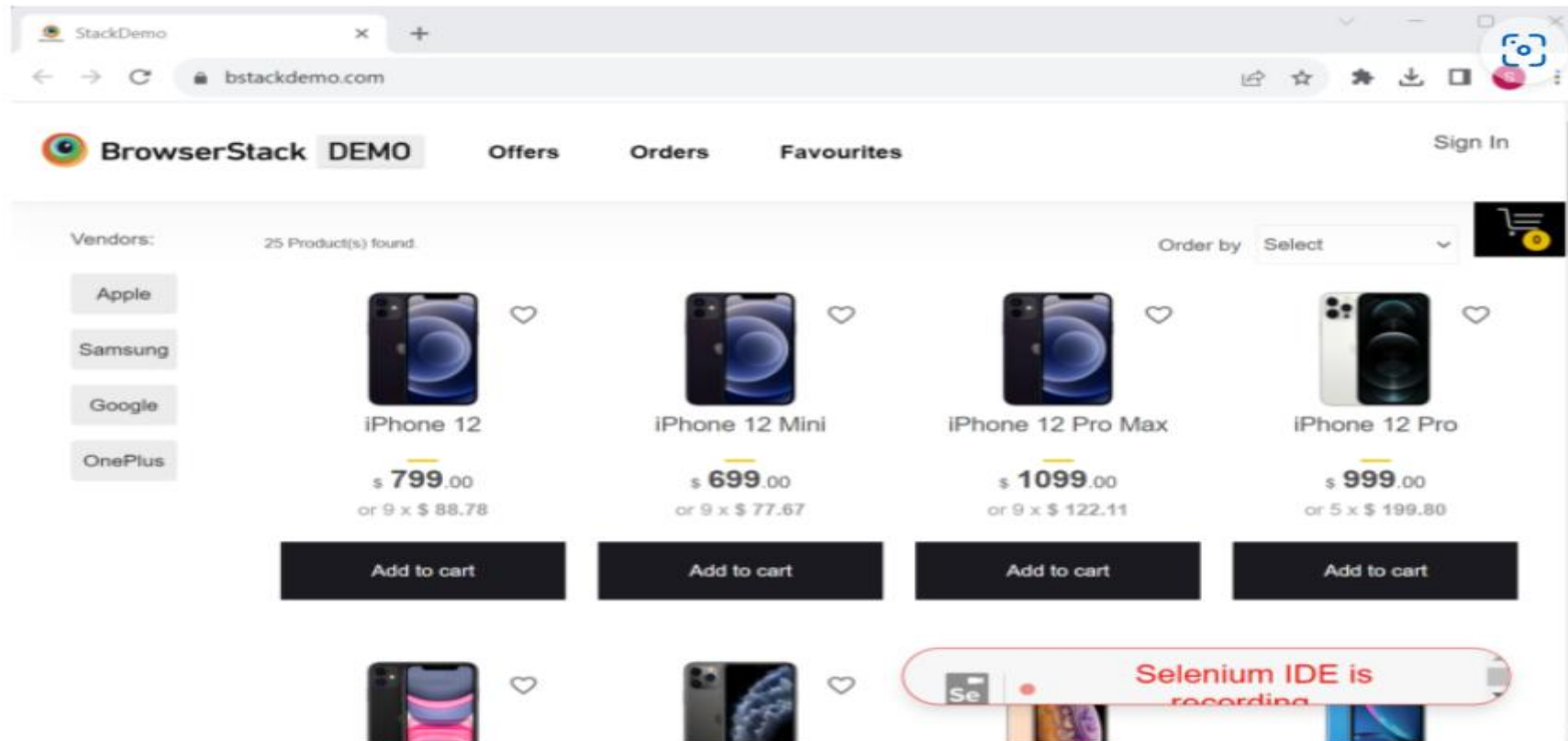
It opens the below pop up as the project is created for the first time.



Why do we need selenium waits?

Record process in Selenium IDE

6. Enter base URL as <https://bstackdemo.com/> and click on the “Start Recording” button.



Why do we need selenium waits?

Record process in Selenium IDE

7. On BStackDemo page, perform the following steps for recording:
 - a. Filter Order by to “Lowest to highest”.
 - b. Click on Samsung model.
 - c. Click “Add to cart” for the first item.
 - d. Click on the “Checkout” button.

Why do we need selenium waits?

Record process in Selenium IDE

8. Go to IDE and click on the Record button to stop the recording.

The screenshot displays the Selenium IDE interface for a project named 'BStackDemo*'. The 'Tests' panel on the left shows a test named 'Checkout*'. The main area contains a table of recorded commands:

	Command	Target	Value
1	open	https://bstackdemo.com/	
2	set window size	1054x656	
3	click	css=select	
4	select	css=select	label=Lowest to highest

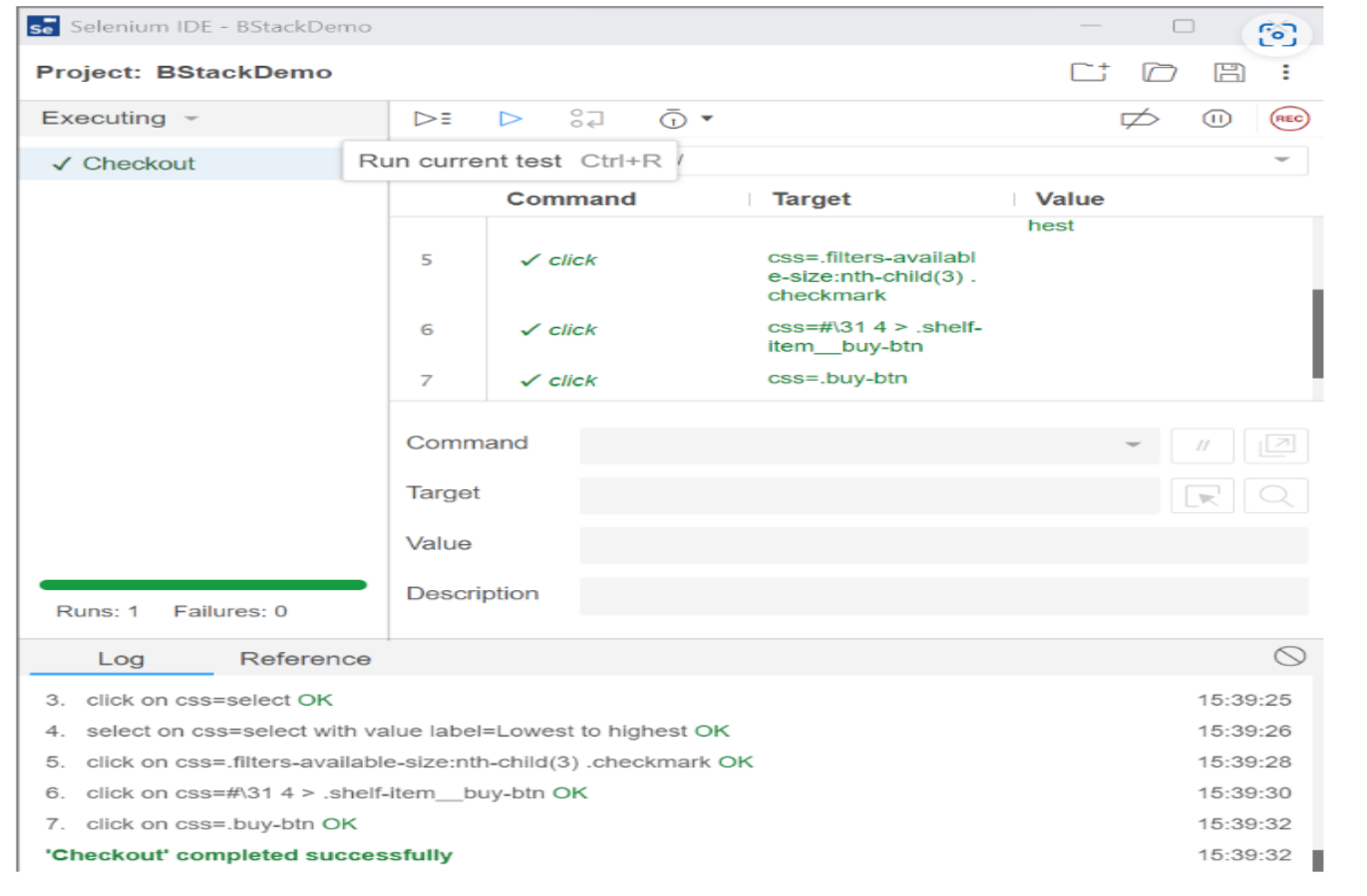
Below the table, there are input fields for 'Command', 'Target', 'Value', and 'Description'. The 'Log' panel at the bottom shows the execution results:

- 3. click on css=select OK 15:25:37
- 4. select on css=select with value label=Lowest to highest OK 15:25:40
- 5. click on css=.filters-available-size:nth-child(3) .checkmark OK 15:25:43
- 6. click on css=#\31 4 > .shelf-item__buy-btn OK 15:25:46
- 7. click on css=.buy-btn OK 15:25:49
- 'Checkout' completed successfully 15:25:50

Why do we need selenium waits?

Playback process in Selenium IDE

1. Click on the “Run current test” button on Selenium IDE to execute the recorded script.



Why do we need selenium waits?

Playback process in Selenium IDE

2. It executes the same steps recorded previously and gives a result as Pass if all the steps are performed correctly. In case it is unable to interact with any element during playback, the execution fails.
3. Logs are captured for each step along with the time taken to execute each step.

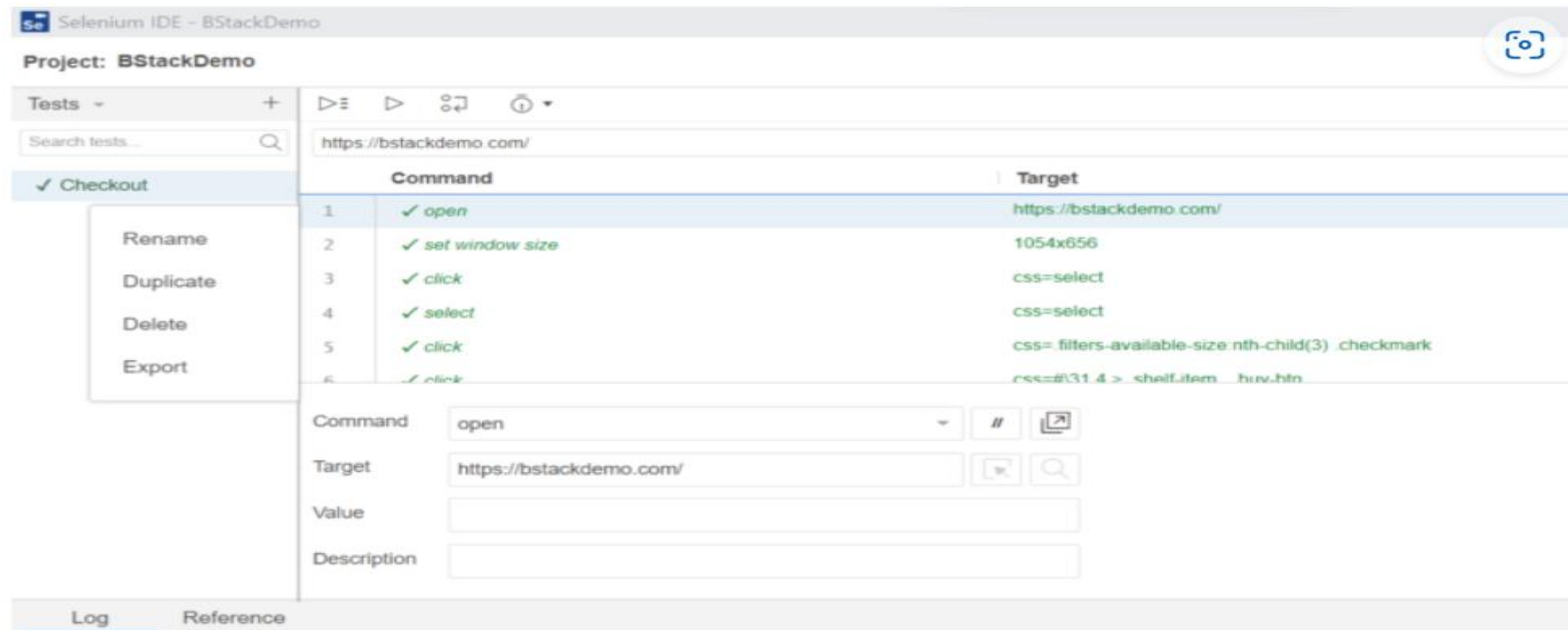
Runs: 1 Failures: 0		Description	
Log	Reference		
Running 'Checkout'			15:39:21
1. open on https://taskadrama.com/ OK			15:39:21
2. setWindowSize on 100x1000 OK			15:39:23
3. click on css=select OK			15:39:25
4. select on css=select with value label=Lowest to highest OK			15:39:26
5. click on css= filters-available-size nth-child(3) checkmark OK			15:39:28
6. click on css=#l31 4 > .shelf-item__buy-btn OK			15:39:30
7. click on css= buy-btn OK			15:39:32
'Checkout' completed successfully			15:39:32

Why do we need selenium waits?

Export Testcases in Selenium IDE

Testcase can be exported in different programming languages from Selenium IDE by following below steps:

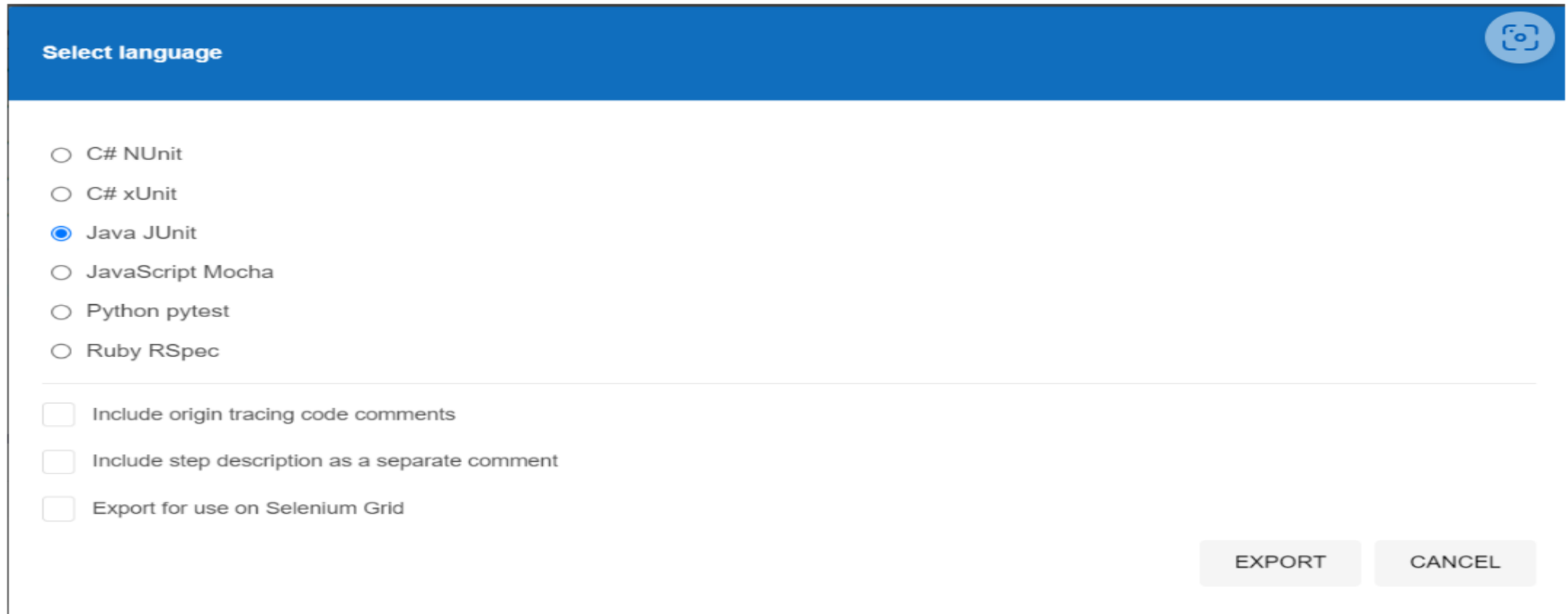
1. Click on the "Export" button for the test case recorded.



Why do we need selenium waits?

Export Testcases in Selenium IDE

2. This opens a popup to Select language. Choose any preferred language. (for eg: Java JUnit) and click on the "Export" button.



Select language

- ☐ C# NUnit
- ☐ C# xUnit
- ☒ Java JUnit
- ☐ JavaScript Mocha
- ☐ Python pytest
- ☐ Ruby RSpec

☐ Include origin tracing code comments

☐ Include step description as a separate comment

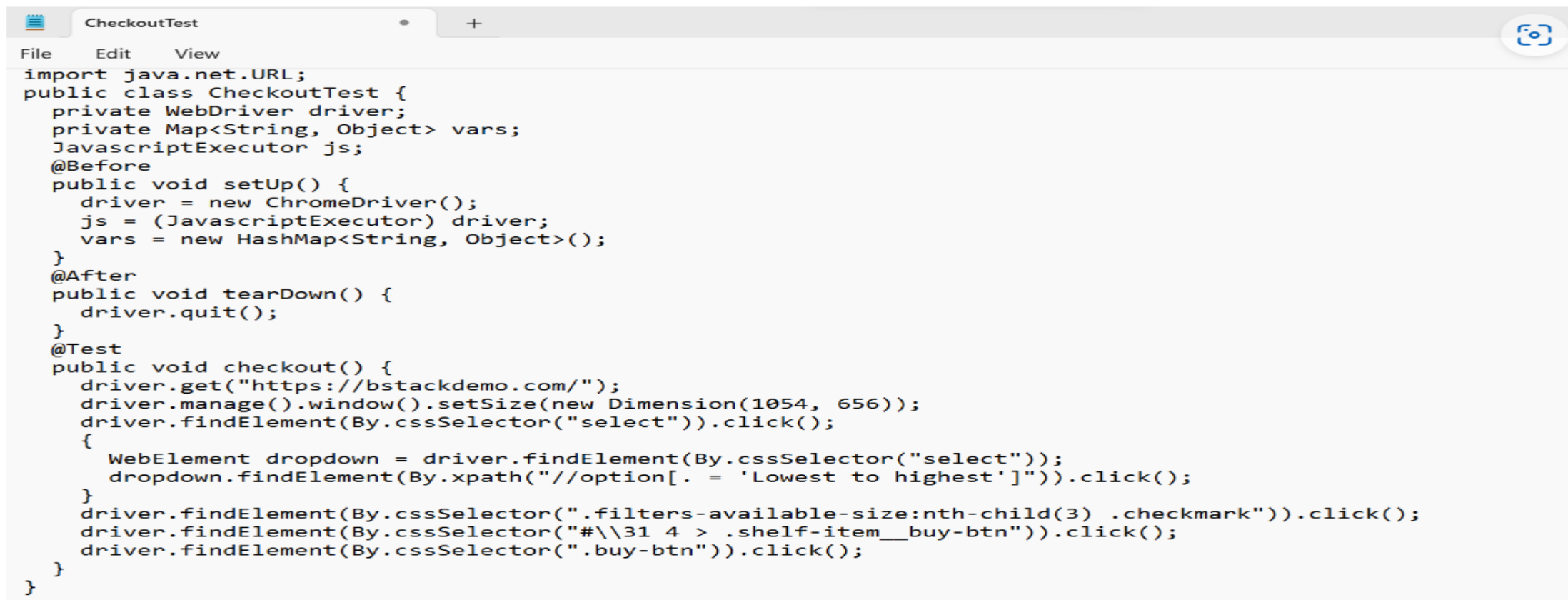
☐ Export for use on Selenium Grid

EXPORT CANCEL

Why do we need selenium waits?

Export Testcases in Selenium IDE

3. Save the file in the local system and later open the file using Notepad or any supported editor to verify the test script.

A screenshot of a code editor window titled 'CheckoutTest'. The editor has a menu bar with 'File', 'Edit', and 'View'. The code is written in Java and uses Selenium WebDriver. It includes imports for java.net.URL, WebDriver, Map, JavascriptExecutor, and HashMap. The class CheckoutTest has three methods: setUp() for initializing the driver and vars, tearDown() for quitting the driver, and checkout() for the main test logic. The checkout() method performs several actions: getting the URL, setting window size, clicking a 'select' button, selecting an option from a dropdown, clicking a checkmark, clicking a buy button, and clicking another buy button.

```
import java.net.URL;
public class CheckoutTest {
    private WebDriver driver;
    private Map<String, Object> vars;
    JavascriptExecutor js;
    @Before
    public void setUp() {
        driver = new ChromeDriver();
        js = (JavascriptExecutor) driver;
        vars = new HashMap<String, Object>();
    }
    @After
    public void tearDown() {
        driver.quit();
    }
    @Test
    public void checkout() {
        driver.get("https://bstackdemo.com/");
        driver.manage().window().setSize(new Dimension(1054, 656));
        driver.findElement(By.cssSelector("select")).click();
        {
            WebElement dropdown = driver.findElement(By.cssSelector("select"));
            dropdown.findElement(By.xpath("//option[. = 'Lowest to highest']")).click();
        }
        driver.findElement(By.cssSelector(".filters-available-size:nth-child(3) .checkmark")).click();
        driver.findElement(By.cssSelector("#\\31 4 > .shelf-item__buy-btn")).click();
        driver.findElement(By.cssSelector(".buy-btn")).click();
    }
}
```



Recording Test Cases

[expleo]

Table of Content

➤ Table of Content :

- Selenium IDE- Creating Test Cases
- Insert Commands
- Executing the Test Script
- Practices

Selenium IDE – Creating Test Cases

➤ Test Cases :

- We will create test cases by inserting selenium commands instead of recording option.
- For this test, we will search a text operation on any publically available search engine (say "Google"). Subsequently, we will create a Login test case in the same test suite.

Table of Content

➤ Table of Content :

- Selenium IDE- Creating Test Cases
- Insert Commands
- Executing the Test Script
- Practices

Insert Commands

➤ Insert Commands :

- Launch Firefox browser.
- Click on the Selenium icon present on the top right corner on your browser.
- It will launch the default interface of Selenium IDE.
- Enter the project name as "Manual Test".
- Enter the test case name as "Search Test".
- Click on the command text box present on the Test Script Editor Box.

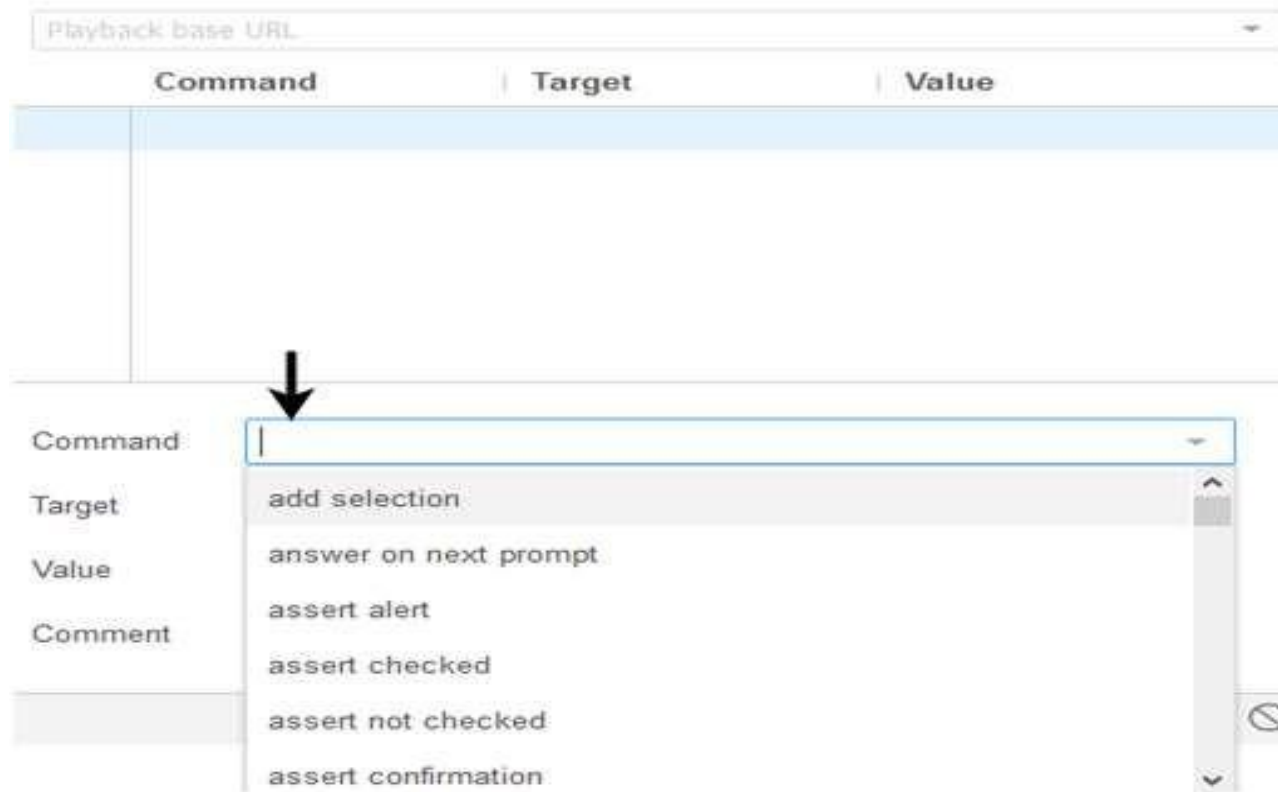
Insert Commands

➤ Insert Commands :

- Modify the properties of First command as:
- Command : open
- Target : <https://www.google.co.in>
- During execution of the test case, this command will load the Google search engine web page on your Firefox browser.

Insert Commands

➤ Insert Commands :



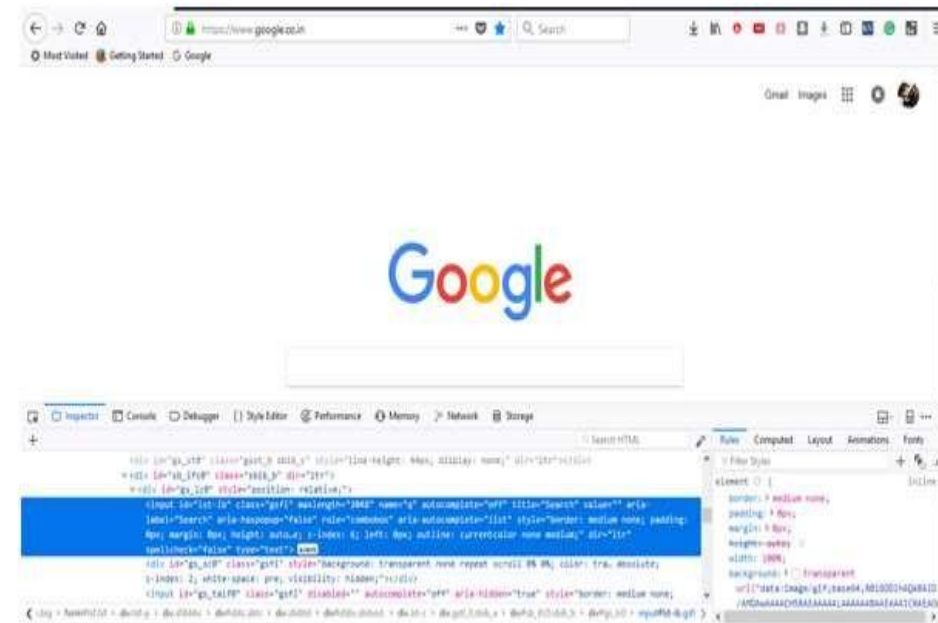
Insert Commands

➤ Insert Commands :

- Now, we have to add a command that will click on the Google search engine text box. For this, we need a unique identification element of the text box which will help the IDE to identify the target location.
- The method for finding a unique identification element involves inspection of HTML codes.
- Open URL: <https://www.google.co.in> your Firefox browser.
- Right click on the Google search text box and select Inspect Element.

Insert Commands

➤ Insert Commands :



Insert Commands

- **Second command as:**
 - Command :click at
 - Target : name=q
 - During execution of the test case, this command will click on the search text box present on the Google search engine web page.

Insert Commands

- **Third command as:**
 - Command : type
 - Target : name=q
 - Value: javaTpointJavaFX tutorial
 - During execution of the test case, this command will type the specified text on the Google search text box.

Insert Commands

➤ Third command as:

- Now add a command which will generate a button click event on our web page. For this event to be generated, we need a unique identification element for the Google search button.
- Right click on the Google search button and select Inspect Element.

Insert Commands

➤ Fourth command as:

- Command : click at
- Target : name=btnK
- During execution of the test case, this command will click on the search button present on the Google
search engine web page.

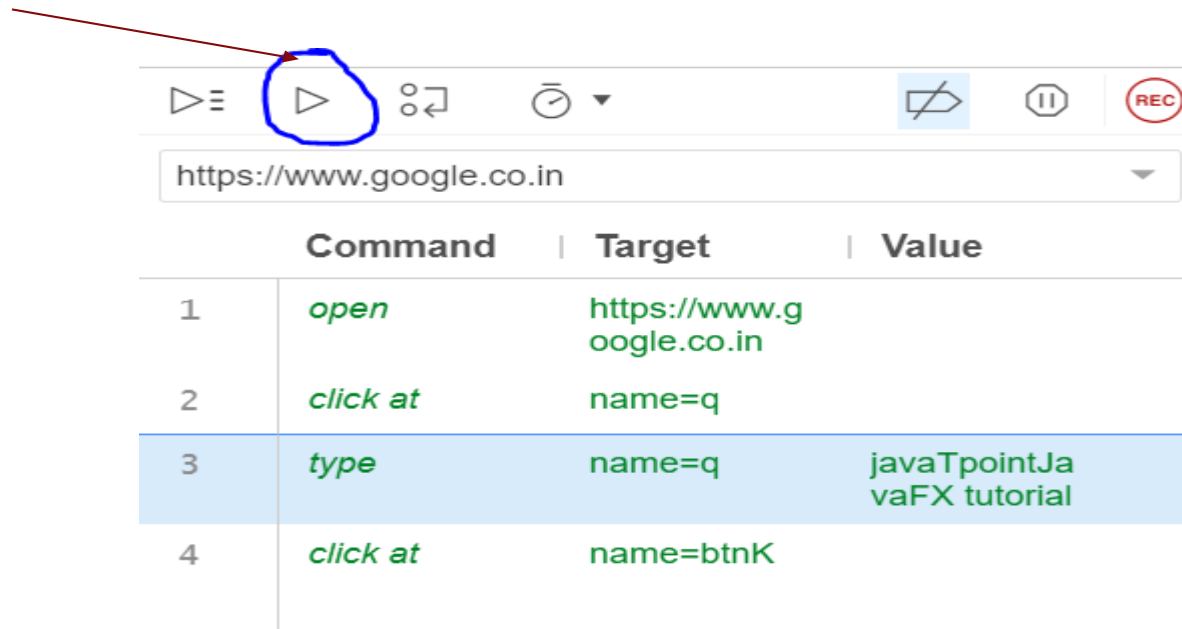
Insert Commands

▶ ▶ ↺ ⌚ ✖ ⏸ REC https://www.google.co.in			
	Command	Target	Value
1	<i>open</i>	https://www.g oogle.co.in	
2	<i>click at</i>	name=q	
3	<i>type</i>	name=q	javaTpointJa vaFX tutorial
4	<i>click at</i>	name=btnK	

Test Script

Executing the Test Script :

- Click on the "Run Current Test" button present on the tool bar menu of the IDE.



Test Script

Executing the Test Script :

The Log pane displays the overall summary of the executed test scripts.

Runs: 1 Failures: 0		Description	
Log	Reference		
1. open on https://www.google.co.in OK			14:58:19
2. clickAt on name=q OK			14:58:19
3. type on name=q with value javaTpointJavaFX tutorial OK			14:58:23
4. clickAt on name=btnK OK			14:58:23
'Test_Hotmail' completed successfully			14:58:23



Introduction to Selenium Commands - Selense

[expleo]

Introduction to Selenese Command

➤ **Selenese Command :**

- Selenese – The language of Selenium or Commands of Selenium
- Selenese consists of Actions, Accessors, Element Locators and Variables

➤ **Actions – Combination of Command, Target and Value**

- **Command : Scripts perform a particular action on a web application**

- Target : Typically take element locator
- Value : Value for any input

Example:

Open - Open a URL

Click – Click button, link, etc...

Type – Type text in text field

Introduction to Selenese Command

- There are only about 6 commands you need to know to be able to automate tests. Same as a manual test case really (open, click, select, type, assert (check), wait).
- **Selenium commands are basically classified in three categories:**
 - **Actions**
 - Actions are the selenium commands that interact directly with the application
 - **Accessors**
 - Accessors are the selenium commands that examine the state of the application and store the results in variables. They are also used to automatically generate Assertions. Allows user to store certain value to user defined variable
 - **Assertions**
 - Assertions are the commands that enable testers to verify the state of the application with the expected state

Actions

➤ Actions:

- Actions are the selenium commands that generally manipulate the state of the application

Command/Syntax	Description
open (url)	It launches the desired URL in the specified browser and it accepts both relative and absolute URLs.
type (locator,value)	It sets the value of an input field, similar to user typing action.
typeKeys (locator,value)	This command simulates keystroke events on the specified element.
click (locator)	This command enables clicks on a link, button, checkbox or radio button.
clickAt (locator,coordString)	This command enables clicks on an element with the help of locator and co-ordinates
doubleClick (locator)	This command enables double clicks on a webelement based on the specified element.
focus (locator)	It moves the focus to the specified element
highlight (locator)	It changes the background color of the specified element to yellow to highlight is useful for debugging purposes.

Actions

➤ Actions:

close()	This command simulates the user clicking the "close" button in the title bar of a popup window or tab.
store (expression,variableName)	This command specifies the name of a variable in which the result is to be stored and expression is the value to store
waitForCondition (script,timeout)	This command executes the specified JavaScript snippet repeatedly until it evaluates to "true".

ACCESSORS

➤ Accessors:

Command/Syntax	Description
storeTitle (variableName)	This command gets the title of the current page.
storeText (locator, variableName)	This command gets the text of an element..
storeValue (locator,variableName)	This command gets the (whitespace-trimmed) value of an input field.
storeTable (tableCellAddress, variableName)	This command gets the text from a cell of a table.
storeLocation (variableName)	This command gets the absolute URL of the current page.
storeElementIndex (locator, variableName)	This command gets the relative index of an element to its parent (starting from 0).
storeBodyText (variableName)	This command gets the entire text of the page.
storeAllButtons (variableName)	It returns the IDs of all buttons on the page.
storeAllFields (variableName)	It returns the IDs of all input fields on the page.
storeAllLinks (variableName)	It returns the IDs of all links on the page.

ACCESSORS

➤ Accessors:

Command/Syntax	Description
storeTitle (variableName)	This command gets the title of the current page.
storeText (locator, variableName)	This command gets the text of an element..
storeValue (locator,variableName)	This command gets the (whitespace-trimmed) value of an input field.
storeTable (tableCellAddress, variableName)	This command gets the text from a cell of a table.
storeLocation (variableName)	This command gets the absolute URL of the current page.
storeElementIndex (locator, variableName)	This command gets the relative index of an element to its parent (starting from 0).
storeBodyText (variableName)	This command gets the entire text of the page.
storeAllButtons (variableName)	It returns the IDs of all buttons on the page.
storeAllFields (variableName)	It returns the IDs of all input fields on the page.
storeAllLinks (variableName)	It returns the IDs of all links on the page.

Practice Program - 1

Open the Page: <https://demo.opencart.com/>

- Click on the Phones & PDAs
- Click on HTC Touch HD
- Extract the Text "HTC Touch HD" from the loaded page and store it in the variable and display

Command	Target	Value
<i>open</i>	https://demo.opencart.com/	
<i>set window size</i>	1382x744	
<i>click</i>	linkText=Phones & PDAs	
<i>click</i>	linkText=HTC Touch HD	
<i>store text</i>	css=h1	x
<i>echo</i>	\${x}	

Practice Program - 2

Open the Page: <https://demo.opencart.com/>

- Store the Title and URL of the Page
- Use Assert method to check the expected URL, Title and Actual Value are same.

Solution :

Command	Target	Value
✓ <i>open</i>	https://demo.opencart.com/	
✓ <i>store title</i>		title
✓ <i>echo</i>	\${title}	
✓ <i>assert title</i>	Your Store	
✓ <i>execute script</i>	return window.location.href	urltext
✓ <i>assert</i>	urltext	https://demo.opencart.com/

Practice Program - 3

- Open the Page: <https://demo.opencart.com/>
- Click on the Phones & PDAs
- Click on HTC Touch HD and click on add to cart.
- Use Assert method to confirm that the product is successfully added to the cart

Solution :

Command	Target	Value
<i>open</i>	/	
<i>click</i>	linkText=Phones & PDAs	
<i>click</i>	linkText=HTC Touch HD	
<i>click</i>	id=button-cart	
<i>assert element present</i>	id=alert	

Selenium IDE Java Script Commands

Command	Target	Value	Comment
Do...Repeat If	Javascript to evaluate		Similar to while, the the first "if" check is done at the end of the loop.
gotof	Javascript to evaluate	Label	(Deprecated) If the expression evaluates to TRUE the execution jumps to LABEL, otherwise continues with the next command. Often used with !statusOK .
gotoLabel	Label		(Deprecated) Jumps to LABEL.
forEach...end	Javascript to evaluate		Loop over the content of an Javascript array.
if-elseif-else-end	Javascript to evaluate		The classic if-then-else conditional. If the expression evaluates to TRUE the "then" section is executed, otherwise the "else" section is executed. Often used with !statusOK
label	Label		(Deprecated) Defines the LABEL position for gotof and gotoLabel to jump to.
times...end	Number		Loop "Number" times.
while...end	Javascript to evaluate		Executes the section between while...end as often/as long as the conditional statement evaluates to true.

Selenium IDE Java Script Commands

Javascript (executeScript_Sandbox recommended)	Result
<code>return Math.floor(Math.random()*11)</code>	Random number between 0 and 10
<code>return Number (\${i}) + 1;</code>	Increase the value of i by one (very useful inside loops)
<code>var x="abc"; return x.length;</code>	Returns the length of the string (here 3).
<code>x="\${myvar}"; return x.length;</code>	Same as above but with variable as input.
<code>var d=new Date(); return d.getDate()+'-'+((d.getMonth()+1))+'-'+d.getFullYear();</code>	Get todays date
<code>var d= new Date(); var m=((d.getMonth()+1)<10)?'0'+(d.getMonth()+1):(d.getMonth()+1); return d.getFullYear()+"-"+m+"-"+d.getDate();</code>	Get todays date in YYYY-MM-DD format
<code>var d= new Date(); var month = d.toLocaleString('default', { month: 'long' }); return month+"-"+d.getDate() + "-" + d.getFullYear();</code>	Get todays date in MMM-DD-YYYY where MMM is the month's name e. g. May-1-2022
<code>var d= new Date(new Date().getTime() + 24 * 60 * 60 * 1000 * 5); var m=((d.getMonth()+1)<10)?'0'+(d.getMonth()+1):(d.getMonth()+1); m=d.getFullYear()+"-"+m+"-"+d.getDate(); return m</code>	Get yesterday's date (-1), tomorrows date (1) or the date in 5 days (5) => Just change the number 5 in the code snippet.
<code>return parseFloat(\$!runtime)-parseFloat(\${starttime});</code>	Calculate the difference between the current runtime and the start time. Very useful for measuring website performance. <i>!runtime</i> is a built-in variable, and <i>starttime</i> is a regular variable, we can fill it with <i>!runtime</i> e. g. at the macro start.

Practice Program 5

Write selenium ide javascript commands to print value from 1 to 5 using while loop

Solution :

Command	Target	Value
<i>store</i>	1	i
<i>while</i>	$\${i} \leq 5$	
<i>echo</i>	$i = \${i}$	
<i>execute script</i>	return Number ($\${i}$)+1	i
<i>end</i>		

Practice Program 6

Write selenium ide javascript commands to ,

- Verify whether the Availability of the stock of the mac book in <https://demo.opencart.com/>
- Verify the availability by visiting page at least 3 times continuously

Solution :

Command	Target	Value
<i>open</i>	https://demo.opencart.com/	
<i>store</i>	1	i
<i>while</i>	`\${i}<=3`	
<i>click</i>	linkText=MacBook	
<i>assert text</i>	css=.col-sm li:nth-child(4)	Availability: In Stock
<i>execute script</i>	history.back()	
<i>execute script</i>	return Number(`\${i}`)+1	i
<i>end</i>		
<i>echo</i>	Executed `\${i}`	

and

open

//



https://demo.opencart.com/





Summary

- Need for Automation Testing
- Steps involved in choosing the automation framework.
- How to start with Selenium IDE.

THANK YOU